LAPORAN PRAKTIKUM 1-5
PEMOGRAMAN BERORIENTASI OBJEK



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu:
Nurfiah, S.St, M.Kom

Asisten Praktikum:
Muhammad Dawi Syauqi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

PRAKTIKUM 1

A. Langkah-Langkah

1. Buat Class Rekening

a. Buat Class Rekening

public class Rekening {
Class ini menjadi blueprint untuk objek rekening. Didalamnya kita menyimpan data rekening dan method untuk melakukan aktivitas terhadap rekening.

b. Tambahkan State/Atribut

String nomorRekening;
Atribut ini digunakan untuk menyimpan nomor rekening. Karena nomor rekening bisa mengandung angka yang tidak digunakan untuk perhitungan, String lebih sesuai daripada int.

String namaPemilik;
Menyimpan nama pemilik rekening.

double saldo;
Menyimpan saldo rekening. Digunakan double karena nilai uang yang digunakan dalam program dapat memiliki nilai desimal.

c. Buat Constructor

public Rekening(String nomor, String nama, double saldoAwal) {
constructor digunakan untuk memberikan kondisi awal atau state awal objek ketika objek dibuat.

nomorRekening = nomor;
Memasukkan nilai parameter nomor ke atribut nomorRekening.

namaPemilik = nama;
Memasukkan nama yang diberikan ketika objek dibuat ke atribut namaPemilik.

saldo = saldoAwal;
Mengisi saldo rekening dengan saldo awal.

System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
Menampilkan informasi bahwa rekening berhasil dibuat. Tanda + digunakan untuk menggabungkan teks dengan nilai variabel.

d. Buat Method/Behavior

public void setorTunai(double nominal) {
Membuat method bernama setorTunai. Method menerima parameter nominal, yaitu jumlah uang yang ingin disetorkan.

if (nominal > 0) {
Memeriksa apakah nominal setor lebih besar dari 0. Kalau lebih dari 0, transaksi dilanjutkan.

saldo += nominal;
Menambahkan nominal setor ke saldo.

System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
Menampilkan pesan bahwa setor tunai berhasil sekaligus menampilkan saldo terbaru.

} else { *System.out.println("Gagal: Nominal setor harus lebih dari 0!");*

}
Jika nominal setor tidak lebih dari 0, program menampilkan pesan gagal. Program tidak langsung berhenti atau error.

public void cekInformasi() {
Membuat method untuk menampilkan informasi rekening.

System.out.println("----- INFO REKENING -----");
Menampilkan judul informasi rekening.

System.out.println("No. Rekening : " + nomorRekening);
Menampilkan nomor rekening.

System.out.println("Nama Pemilik : " + namaPemilik);
Menampilkan nama pemilik.

System.out.println("Saldo Akhir : Rp" + saldo);
Menampilkan saldo terakhir.

System.out.println("--------------------------");
Menampilkan garis pembatas.

2. Membuat Engine CLI pada Main.java
    a. Import dan Class

import java.util.Scanner;
Digunakan untuk mengimpor class Scanner dari Java. Scanner diperlukan agar program bisa menerima input dari keyboard, misalnya ketika user memasukkan nomor rekening, nama pemilik, saldo, dan pilihan menu.

public class Main {

Membuat class bernama Main. Class Main digunakan sebagai class utama yang menjalankan program.

b. Method main() dan Inisialisasi

public static void main(String[] args) {
main() adalah method utama yang akan dijalankan pertama kali ketika program Java dijalankan.

Scanner input = new Scanner(System.in);
Membuat objek Scanner bernama input. System.in menunjukkan bahwa sumber input berasal dari keyboard.

Rekening akunAktif = null;
Membuat variabel akunAktif bertipe Rekening. Variabel ini nantinya digunakan untuk menyimpan objek rekening yang sedang aktif. Nilai awalnya null, artinya belum ada objek rekening yang dibuat.

boolean isRunning = true;
Membuat variabel bertipe boolean bernama isRunning.

c. Judul Program

System.out.println("=== SISTEM PERBANKAN MINI ===");
Menampilkan judul program pada console.

d. Perulangan Menu

while (isRunning) {
Membuat perulangan while. Selama isRunning bernilai true, seluruh menu di dalam while akan terus dijalankan. Ketika user memilih menu 0, nilai isRunning akan diubah menjadi false, sehingga perulangan berhenti.

System.out.println("\nMenu Utama:");
Menampilkan tulisan Menu Utama. \n digunakan untuk membuat baris baru sebelum tulisan.

System.out.println("1. Buka Rekening Baru");
Menampilkan pilihan menu nomor 1 untuk membuka rekening baru.

System.out.println("2. Setor Tunai");
Menampilkan pilihan menu nomor 2 untuk melakukan setor tunai.

System.out.println("3. Tarik Tunai");
Menampilkan pilihan menu nomor 3 untuk tarik tunai.

System.out.println("4. Cek Informasi Rekening");
Menampilkan pilihan menu nomor 4 untuk melihat informasi rekening.

System.out.println("0. Keluar");
Menampilkan pilihan nomor 0 untuk keluar dari program.

System.out.print("Pilih menu: ");
Meminta user memilih menu.

e. Membaca Pilihan Menu

int pilihan = input.nextInt();
Membaca angka yang dimasukkan user dan menyimpannya ke variabel pilihan.

input.nextLine();
Digunakan untuk membersihkan sisa Enter setelah nextInt().

f. Percabangan switch

switch (pilihan) {
Digunakan untuk menentukan kode mana yang dijalankan berdasarkan nilai pilihan.

g. Case 1 – Buka Rekening Baru

case 1:
Bagian ini dijalankan ketika user memilih menu:

System.out.print("Masukkan No Rekening: ");
Meminta user memasukkan nomor rekening.

String no = input.nextLine();
Membaca nomor rekening yang dimasukkan user dan menyimpannya ke variabel no.

System.out.print("Masukkan Nama Pemilik: ");
Meminta nama pemilik rekening.

String nama = input.nextLine();
Membaca nama pemilik dan menyimpannya ke variabel nama.

System.out.print("Masukkan Saldo Awal: ");
Meminta user memasukkan saldo awal rekening.

double saldo = input.nextDouble();
Membaca saldo awal yang dimasukkan user dan menyimpannya ke variabel saldo. Digunakan tipe double karena saldo merupakan nilai numerik yang bisa memiliki angka desimal.

akunAktif = new Rekening(no, nama, saldo);

new Rekening(no, nama, saldo) digunakan untuk membuat objek baru dari class Rekening.

break;
Menghentikan proses case 1. Program kemudian kembali ke perulangan while dan menampilkan menu utama lagi.

h. Case 2 – Setor Tunai

case 2:
Dijalankan ketika user memilih menu setor tunai.

if (akunAktif == null) {
Mengecek apakah belum ada rekening aktif. Kalau akunAktif masih null, berarti user belum membuat rekening.

System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
Menampilkan pesan error jika user mencoba setor tetapi belum mempunyai rekening.

} else {
Bagian else dijalankan jika akunAktif tidak null. Artinya user sudah mempunyai rekening aktif.

System.out.print("Masukkan nominal setor: ");
Meminta nominal uang yang ingin disetor.

double setor = input.nextDouble();
Membaca nominal setor dan menyimpannya ke variabel setor.

akunAktif.setorTunai(setor);

Memanggil method setorTunai() yang terdapat pada class Rekening. Nilai setor diberikan sebagai parameter.

break;
Menghentikan case 2 dan kembali ke menu utama.

i. Case 3 – Tarik Tunai

case 3:
Dijalankan ketika user memilih menu tarik tunai.

System.out.println("Fitur ini akan dikerjakan sebagai Tugas Mandiri.");
Menampilkan Fitur ini akan dikerjakan sebagai Tugas Mandiri

break;
Menghentikan case 3 dan kembali ke menu utama.

j. Case 4 – Cek Informasi Rekening

case 4:
Dijalankan ketika user memilih menu:

if (akunAktif == null) {
Memeriksa apakah user sudah memiliki rekening aktif.

System.out.println("Error: Anda belum membuka rekening!");
Jika belum ada rekening, program menampilkan pesan tersebut.

} else {
Jika akunAktif tidak null, berarti rekening sudah tersedia.

akunAktif.cekInformasi();

Memanggil method cekInformasi() dari objek rekening aktif.

break;
Menghentikan case 4.

k. Case 0 – Keluar

case 0:
Dijalankan ketika user memilih menu 0.

isRunning = false;
Mengubah nilai dari true ke false, maka setelah menjadi false, perulangan berhenti.

System.out.println("Sistem ditutup. Terima kasih!");
Menampilkan pesan bahwa program telah ditutup.

break;
Menghentikan case 0.

l. Default

default:
Dijalankan jika user memasukkan pilihan yang tidak tersedia.

System.out.println("Pilihan tidak valid.");
Menampilkan pesan bahwa pilihan user tidak valid.

m. Menutup Program

input.close();
Menutup objek Scanner setelah program selesai digunakan.

B. Tugas/Latihan Praktikum

1. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan : Saldo tidak boleh menjadi minus dan minimal nominal Tarik Tunai adalah 10.000. jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: “Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]” atau Transaksi Gagal : minimal nominal penarikan 10.000”.
- Terapkan method ini dengan menu nomor 3 pada class main.

a. Class Rekening.

```
public void tarikTunai(double nominal) {
	if (nominal < 10000) {
		System.out.println("Transaksi Gagal: Minimal penarikan RP 1"
				+ "10.000");
	} else if (nominal > saldo){
		System.out.println("Transaksi Gagal: Saldo tidak mencukupi. saldo anda Rp " + saldo );
	} else {
		saldo -= nominal;
		System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
	}
}
```

b. Class Main

```
case 3:
	if (akunAktif == null) {
	System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
	} else {
```

```
        System.out.print("Masukkan nominal tarik tunai: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
}
        break;
```

Penjelasan Code:

a. Class Rekening

public void tarikTunai(double nominal) {
Membuat method bernama tarikTunai yang digunakan untuk melakukan transaksi penarikan uang.

if (nominal < 10000) {
Mengecek apakah nominal penarikan kurang dari Rp10.000. Jika benar, berarti penarikan tidak memenuhi batas minimal yang ditentukan.

System.out.println("Transaksi Gagal: Minimal penarikan RP10.000");
Menampilkan pesan kegagalan transaksi ke console dan memberi tahu bahwa ada batas minimal penarikan.

} else if (nominal > saldo){
Jika nominal tidak kurang dari Rp10.000, program mengecek kondisi berikutnya. Kondisi ini mengecek apakah nominal yang ingin ditarik lebih besar daripada saldo yang tersedia.

System.out.println("Transaksi Gagal: Saldo tidak mencukupi. saldo anda Rp" + saldo);
Menampilkan pesan bahwa saldo tidak mencukupi.

} else {

Jika dua kondisi sebelumnya tidak terpenuhi, program masuk ke bagian else.

saldo -= nominal;
Mengurangi saldo dengan nominal yang ditarik.

System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
Menampilkan pesan bahwa penarikan berhasil dan menampilkan saldo setelah transaksi.

b. Main Class

case 3:
case 3 dijalankan ketika pengguna memilih menu nomor 3, yaitu Tarik Tunai.

if (akunAktif == null) {
Kode ini mengecek apakah akunAktif masih bernilai null. null berarti belum ada objek rekening yang digunakan sebagai rekening aktif. Jadi program memastikan pengguna sudah mempunyai rekening sebelum melakukan tarik tunai.

System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
Jika akunAktif == null, program menampilkan pesan bahwa pengguna belum memiliki nomor rekening.

} else {
else dijalankan jika akunAktif tidak bernilai null. Artinya, pengguna sudah memiliki rekening aktif dan boleh melanjutkan proses tarik tunai.

System.out.print("Masukkan nominal tarik tunai: ");
Menampilkan perintah agar pengguna memasukkan jumlah uang yang ingin ditarik.

double tarik = input.nextDouble();
input.nextDouble() membaca angka yang dimasukkan pengguna.

akunAktif.tarikTunai(tarik);
bagian yang menghubungkan Main.java dengan method tarikTunai() yang ada di class Rekening.

break;
break digunakan untuk menghentikan case 3.

Output

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih Menu: 3
Masukkan nominal tarik tunai: 5000
Transaksi Gagal: Minimal penarikan RP10.000

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih Menu: 3
Masukkan nominal tarik tunai: 110000
Transaksi Gagal: Saldo tidak mencukupi. saldo anda Rp100000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih Menu: 3
Masukkan nominal tarik tunai: 20000
Tarik tunai Rp20000.0 berhasil. Saldo saat ini: Rp80000.0
```

2. Tantangan Multi-Akun

   Saat ini, aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;). Jika pengguna memilih menu “Buka Rekening Baru” untuk kedua kalinya, akan lama akan tertimpa.

   - Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList<Rekening>).
   - Ubah logika menu “Buka Rekening”: setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.
   - (Bonus): Tambahkan menu baru untuk “Ganti Akun”, di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam Arraylist untuk dijadikan akunAktif.

```
import java.util.ArrayList;
import java.util.Scanner;
public class Main_2511532024 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening_2511532024> daftarRekening = new
ArrayList<>();
        Rekening_2511532024 akunAktif = null;
        boolean isRunning = true;
        System.out.println("=== SISTEM PERBANKAN MINI
===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
```

```
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu: ");
            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldo = input.nextDouble();
                    input.nextLine();
                    Rekening_2511532024 rekeningBaru =
                            new Rekening_2511532024(no, nama, saldo);
                    daftarRekening.add(rekeningBaru);
                    akunAktif = rekeningBaru;
                    System.out.println("Rekening berhasil
ditambahkan.");
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println(
                                "Error: Mohon maaf, Anda belum memiliki
nomor rekening!");
                    } else {
                        System.out.print("Masukkan nominal setor: ");
                        double setor = input.nextDouble();
                        akunAktif.setorTunai(setor);
                    }
```

```
                break;

            case 3:
                if (akunAktif == null) {
                    System.out.println(
                            "Error: Mohon maaf, Anda belum memiliki
nomor rekening!");
                } else {
                    System.out.print("Masukkan nominal tarik tunai: ");
                    double tarik = input.nextDouble();
                    akunAktif.tarikTunai(tarik);
                }
                break;



            case 4:
                if (akunAktif == null) {
                    System.out.println(
                            "Error: Anda belum membuka rekening!");
                } else {
                    akunAktif.cekInformasi();
                }
                break;

            case 5:
                if (daftarRekening.isEmpty()) {
                    System.out.println(
                            "Error: Belum ada rekening yang tersedia!");
                } else {
                    System.out.print(
                            "Masukkan nomor rekening yang ingin
```

```
digunakan: ");
                String nomorCari = input.nextLine();
                boolean ditemukan = false;
                for (Rekening_2511532024 rekening :
daftarRekening) {
                    if (rekening.nomorRekening.equals(nomorCari))
{
                        akunAktif = rekening;
                        ditemukan = true;
                        System.out.println(Akun berhasil diganti ke
rekening "
                                + rekening.nomorRekening);
                        break;
                    }
                }
                if (!ditemukan) {
                    System.out.println(
                            "Error: Nomor rekening tidak ditemukan!");
                }
            }
            break;

        case 0:
            isRunning = false;
            System.out.println("Sistem ditutup. Terima kasih!");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
      }
    }
    input.close();
```

```
    }
}
```

Penjelasan Code:

import java.util.ArrayList;
ArrayList digunakan untuk menyimpan lebih dari satu objek rekening.

ArrayList<Rekening_2511532024> daftarRekening = new ArrayList<>();
daftarRekening menjadi tempat penyimpanan seluruh rekening yang dibuat.

Rekening_2511532024 rekeningBaru = new Rekening_2511532024(no, nama, saldo);
Membuat objek rekening baru berdasarkan data nomor rekening, nama pemilik, dan saldo awal.

daftarRekening.add(rekeningBaru);
Menambahkan objek tersebut ke ArrayList.

case 5:
Menentukan bahwa kode berikutnya dijalankan ketika pengguna memilih menu 5, yaitu Ganti Akun.

if (daftarRekening.isEmpty()) {
Mengecek apakah daftarRekening kosong atau belum memiliki rekening.

System.out.println("Error: Belum ada rekening yang tersedia!");

Jika ArrayList kosong, program menampilkan pesan bahwa belum ada rekening yang bisa dipilih.

} else {
Jika ArrayList tidak kosong, program menjalankan kode di dalam else.

System.out.print("Masukkan nomor rekening yang ingin digunakan: ");
Meminta pengguna memasukkan nomor rekening yang ingin dijadikan akun aktif.

String nomorCari = input.nextLine();
Menyimpan nomor rekening yang dimasukkan pengguna ke dalam variabel nomorCari.

boolean ditemukan = false;
Membuat variabel ditemukan untuk menandai apakah rekening yang dicari ditemukan. Nilai awalnya false karena program belum menemukan rekening tersebut.

for (Rekening_2511532024 rekening : daftarRekening) {
Melakukan perulangan untuk memeriksa setiap rekening yang ada di ArrayList.

if (rekening.nomorRekening.equals(nomorCari)) {
Membandingkan nomor rekening yang sedang diperiksa dengan nomor rekening yang dimasukkan pengguna.

akunAktif = rekening;
Jika rekening ditemukan, objek rekening tersebut dijadikan akun

yang sedang aktif.

ditemukan = true;
Mengubah nilai ditemukan dari false menjadi true

System.out.println("Akun berhasil diganti ke rekening " + rekening.nomorRekening);
Menampilkan pesan bahwa akun berhasil diganti.

break;
Menghentikan perulangan for karena rekening yang dicari sudah ditemukan.

if (!ditemukan) {
Mengecek apakah rekening tidak ditemukan. Tanda ! berarti kebalikan dari nilai boolean.

System.out.println("Error: Nomor rekening tidak ditemukan!");
Jika sampai seluruh rekening diperiksa tetapi tidak ada yang cocok, program menampilkan pesan bahwa nomor rekening tidak ditemukan.

break;
Menghentikan case 5 agar program kembali ke menu utama, bukan lanjut menjalankan case berikutnya.

Output:

```
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 001
Masukkan Nama Pemilik: nisa
Masukkan Saldo Awal: 1000000
Rekening atas nama nisa berhasil dibuat dengan saldo Rp1000000.0
Rekening berhasil ditambahkan.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 002
Masukkan Nama Pemilik: jj
Masukkan Saldo Awal: 3000000
Rekening atas nama jj berhasil dibuat dengan saldo Rp3000000.0
Rekening berhasil ditambahkan.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih Menu: 5
Masukkan nomor rekening yang ingin digunakan: 002
Akun berhasil diganti ke rekening 002
```

PRAKTIKUM 2

A. Langkah-Langkah

1. Buat Class Diagram Transaksi

   a. Buat Class dengan nama Transaksi

public class Transaksi_2511532024 {
Baris ini digunakan untuk membuat sebuah *class* bernama *Transaksi_2511532024*. Kata *public* berarti *class* ini dapat diakses oleh *class* lain.

   b. Tambahkan State/Atribut dan Constructor

String idTransaksi;
Baris ini membuat atribut *idTransaksi* dengan tipe data *String*. Atribut tersebut digunakan untuk menyimpan identitas atau kode dari setiap transaksi.

String jenis;
Baris ini membuat atribut *jenis* dengan tipe data *String*. Atribut ini digunakan untuk menyimpan jenis transaksi.

double nominal;
Baris ini membuat atribut *nominal* dengan tipe data *double*. Atribut tersebut digunakan untuk menyimpan jumlah uang dalam transaksi.

public Transaksi_2511532024(String id, String jenis, double nominal) {
Baris ini membuat *constructor* untuk *class Transaksi_2511532024*. *Constructor* digunakan untuk memberikan nilai awal pada atribut ketika objek *Transaksi_2511532024* dibuat. Parameter *id* digunakan untuk menerima *ID* transaksi, jenis menerima jenis transaksi, sedangkan nominal menerima jumlah uang transaksi.

this.idTransaksi = id;
Baris ini memasukkan nilai dari parameter *id* ke atribut *idTransaksi*. *this.idTransaksi* menunjukkan atribut yang berada di dalam objek, sedangkan *id* merupakan parameter yang diterima oleh *constructor.*

this.jenis = jenis;
Baris ini memasukkan nilai dari parameter jenis ke atribut *jenis*. Penggunaan *this.jenis* menunjukkan bahwa jenis yang berada di sebelah kiri adalah atribut milik objek, sedangkan jenis di sebelah kanan merupakan parameter *constructor.*

this.nominal = nominal;
Baris ini memasukkan nilai dari parameter nominal ke atribut *nominal*. Dengan demikian, jumlah transaksi yang diberikan ketika objek dibuat akan disimpan di dalam objek tersebut.

c. Buat Method

public void cetakDetail() {
Baris ini membuat method bernama *cetakDetail().* Method ini digunakan untuk menampilkan informasi dari sebuah objek transaksi. Kata *void* menunjukkan bahwa method ini tidak mengembalikan nilai.

System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
Baris ini digunakan untuk menampilkan detail transaksi ke layar. *System.out.println()* berfungsi mencetak teks, sedangkan tanda + digunakan untuk menggabungkan teks dengan nilai dari atribut *idTransaksi*, *jenis*, dan *nominal.*

2. Membuat Relasi pada Rekening.java

ArrayList<Transaksi_2511532024> riwayatTransaksi;
Baris ini membuat atribut bernama *riwayatTransaksi* dengan tipe *ArrayList<Transaksi_2511532024>*. Artinya, *riwayatTransaksi* digunakan untuk menyimpan banyak objek *Transaksi_2511532024* dalam satu rekening.

public Rekening_2511532024(String nomor, String nama, double saldoAwal) {
Baris ini merupakan *constructor* dari *class Rekening_2511532024*. *Constructor* ini dijalankan ketika objek rekening baru dibuat. Parameter nomor digunakan untuk nomor rekening, nama untuk nama pemilik, dan saldoAwal untuk saldo awal.

nomorRekening = nomor;
Baris ini memasukkan nilai parameter nomor ke atribut *nomorRekening*, sehingga nomor rekening yang diberikan akan disimpan dalam objek rekening.

namaPemilik = nama;
Baris ini memasukkan nilai parameter nama ke atribut *namaPemilik,* sehingga nama pemilik rekening tersimpan dalam objek.

saldo = saldoAwal;
Baris ini memasukkan nilai parameter *saldoAwal* ke atribut *saldo*. Dengan demikian, rekening memiliki saldo awal sesuai nilai yang dimasukkan.

this.riwayatTransaksi = new ArrayList<>();
Baris ini membuat *ArrayList* baru untuk menyimpan riwayat transaksi. *new ArrayList<>()* membuat daftar yang awalnya kosong. Nantinya objek *Transaksi_2511532024* yang dibuat saat

transaksi berhasil akan disimpan di dalam daftar tersebut.

System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
Baris ini menampilkan pesan bahwa rekening berhasil dibuat. Nilai *namaPemilik* digabungkan dengan teks sehingga nama pemilik ikut ditampilkan.

3. Mencatat Riwayat Transaksi di Method setorTunai

public void setorTunai(double nominal) {
Baris ini membuat *method setorTunai()* yang digunakan untuk melakukan penyetoran uang ke rekening. Parameter *nominal* bertipe *double* digunakan untuk menerima jumlah uang yang ingin disetor. *void* berarti method ini tidak mengembalikan nilai.

if (nominal > 0) {
Baris ini mengecek apakah nominal yang ingin disetor lebih dari 0. Jika nilainya lebih dari 0, penyetoran dianggap valid dan program akan menjalankan kode di dalam *if.*

saldo += nominal;
Baris ini menambahkan jumlah uang yang disetor ke saldo rekening. Operator += berarti nilai nominal ditambahkan ke nilai saldo yang sudah ada.

String idTrx = "TRX-S-" + System.currentTimeMillis();
Baris ini membuat variabel *idTrx* bertipe *String* untuk menyimpan *ID* transaksi. Teks *"TRX-S-"* digunakan sebagai awalan *ID* untuk transaksi setor, kemudian digabungkan dengan *System.currentTimeMillis()* yang menghasilkan waktu saat ini dalam bentuk angka. Hasilnya digunakan sebagai *ID* transaksi yang berbeda berdasarkan waktu pembuatannya.

Transaksi_2511532024 trxBaru = new Transaksi_2511532024(idTrx, "Kredit", nominal);
Baris ini membuat objek baru bernama *trxBaru* dari *class Transaksi_2511532024. Constructor* menerima tiga nilai, yaitu *idTrx* sebagai *ID* transaksi, *"Kredit"* sebagai *jenis* transaksi karena transaksi tersebut merupakan penyetoran, dan *nominal* sebagai jumlah uang yang disetor.

riwayatTransaksi.add(trxBaru);
Baris ini memasukkan objek *trxBaru* ke dalam *ArrayList* riwayatTransaksi. Dengan demikian, transaksi setor yang berhasil akan tersimpan sebagai riwayat transaksi rekening.

System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
Baris ini menampilkan pesan bahwa penyetoran berhasil sekaligus menampilkan nominal yang disetor dan saldo rekening setelah penyetoran.

} else {
Baris ini menunjukkan bagian yang dijalankan apabila kondisi nominal > 0 bernilai salah. Artinya, nominal yang dimasukkan adalah 0 atau kurang dari 0.

System.out.println("Gagal: Nominal setor harus lebih dari 0!");
Baris ini menampilkan pesan bahwa penyetoran gagal karena nominal yang dimasukkan harus lebih dari 0.

B. Tugas/Latihan Praktikum

1. Integrasi Rekam Jejak Penarikan

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java
- Tambahkan logika perekaman objek transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai “Debit” dan kodenya menggunakan prefiks “TRX-T-“. Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

a. Class Rekening.

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
          System.out.println("Transaksi Gagal: Minimal
penarikan RP10.000");
    } else if (nominal > saldo){
          System.out.println("Transaksi Gagal: Saldo
tidak mencukupi. saldo anda Rp" + saldo );
    } else {
          saldo -= nominal;
          String idTrx = "TRX-T-" +
System.currentTimeMillis();
          Transaksi_2511532024 trxBaru = new
Transaksi_2511532024(idTrx, "Debit", nominal);
          riwayatTransaksi.add(trxBaru);
          System.out.println("Tarik tunai Rp" + nominal
+ " berhasil. Saldo saat ini: Rp" + saldo);
     }
}
```

Penjelasan Code:

a. Class Rekening

public void tarikTunai(double nominal) {
Baris ini merupakan deklarasi *method tarikTunai.* Kata *public* berarti method dapat dipanggil dari *class* lain. *void* berarti method ini tidak mengembalikan nilai. *tarikTunai* adalah nama method yang digunakan untuk proses penarikan uang, sedangkan *double nominal* merupakan parameter yang digunakan untuk menyimpan jumlah

uang yang akan ditarik.

if (nominal < 10000) {
Baris ini menggunakan *if* untuk memeriksa apakah nilai nominal kurang dari 10000. Jika jumlah uang yang ingin ditarik kurang dari Rp10.000, maka kondisi bernilai benar dan program menjalankan perintah di dalam blok *if.*

System.out.println("Transaksi Gagal: Minimal penarikan RP10.000");
Baris ini digunakan untuk menampilkan pesan ke layar menggunakan *System.out.println().* Pesan tersebut memberitahukan bahwa transaksi gagal karena jumlah penarikan yang diperbolehkan minimal adalah Rp10.000.

} else if (nominal > saldo) {
Baris ini menggunakan *else if* untuk melakukan pemeriksaan kedua apabila kondisi *if* sebelumnya tidak terpenuhi. Kondisi *nominal > saldo* digunakan untuk mengecek apakah jumlah uang yang ingin ditarik lebih besar daripada saldo yang tersedia. Jika *nominal* lebih besar dari *saldo*, berarti *saldo* rekening tidak mencukupi.

System.out.println("Transaksi Gagal: Saldo tidak mencukupi. saldo anda Rp" + saldo);
Baris ini menampilkan pesan bahwa transaksi gagal karena saldo tidak mencukupi. Tanda + *saldo* digunakan untuk menggabungkan nilai saldo saat ini dengan teks yang ditampilkan. Jadi, program tidak hanya memberi tahu bahwa saldo tidak cukup, tetapi juga menampilkan jumlah saldo yang tersedia.

} else {
Baris ini merupakan bagian *else* yang dijalankan apabila kedua

kondisi sebelumnya tidak terpenuhi. Artinya, *nominal* penarikan sudah minimal Rp10.000 dan saldo yang dimiliki cukup untuk melakukan penarikan.

saldo -= nominal;
Baris ini digunakan untuk mengurangi saldo berdasarkan jumlah uang yang ditarik. Penulisan *saldo -= nominal* memiliki arti yang sama dengan *saldo = saldo - nominal.*

String idTrx = "TRX-T-" + System.currentTimeMillis();
Baris ini membuat *ID* untuk transaksi. *String* digunakan karena *ID* transaksi berupa teks. Variabel *idTrx* digunakan untuk menyimpan *ID* tersebut. Bagian *"TRX-T-"* menjadi awalan *ID*, sedangkan *System.currentTimeMillis()* menghasilkan waktu saat ini dalam satuan milidetik.

Transaksi_2511532024 trxBaru = new Transaksi_2511532024(idTrx, "Debit", nominal);
Baris ini membuat objek transaksi baru dari *class Transaksi_2511532024*. Objek tersebut disimpan dalam variabel *trxBaru*. *Constructor* menerima tiga data, yaitu *idTrx* sebagai *ID* transaksi, *"Debit"* sebagai jenis transaksi karena terjadi pengurangan saldo, dan nominal sebagai jumlah uang yang ditarik.

riwayatTransaksi.add(trxBaru);
Baris ini digunakan untuk memasukkan objek *trxBaru* ke dalam *riwayatTransaksi*. Dengan begitu, transaksi penarikan yang berhasil akan tersimpan dan nantinya dapat ditampilkan kembali ketika pengguna melihat riwayat atau mutasi rekening.

System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);

Baris ini menampilkan pesan bahwa penarikan berhasil. Nilai nominal digunakan untuk menampilkan jumlah uang yang ditarik, sedangkan nilai saldo digunakan untuk menampilkan saldo setelah penarikan dilakukan.

2. Fitur Cetak Mutasi Rekening
   - Buat sebuah method baru di dalam claa Rekening.java dengan nama cetakMutasi().
   - Di dalam mutasi tersebut, gunakan struktur perulangan foe-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakdetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: “Belum ada Transaksi pada rekening ini”.
   - Buka file Main.java, tambahkan menu baru: “6. Cetak Mutasi (Riwayat)”. Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

   a. Class Rekening

```
public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
          System.out.println("Belum ada transaksi pada
rekening ini");
    } else {
         System.out.println("--- MUTASI REKENING ---");
          for (Transaksi_2511532024 transaksi :
riwayatTransaksi) {
                transaksi.cetakDetail();
                }
          }
```

   b. Class Main

```
case 6:
    akunAktif.cetakMutasi();
break;
```

Penjelasan Code:

a. Class Rekening

public void cetakMutasi() {
Baris ini merupakan deklarasi method *cetakMutasi().* Kata *public* berarti *method* dapat dipanggil dari *class* lain. *void* berarti method tidak mengembalikan nilai. *cetakMutasi* adalah nama *method* yang digunakan untuk menampilkan riwayat atau mutasi transaksi pada rekening.

if (riwayatTransaksi.isEmpty()) {
Baris ini menggunakan *if* untuk mengecek apakah *riwayatTransaksi* masih kosong. *Method .isEmpty()* digunakan untuk mengetahui apakah list tidak memiliki data transaksi. Jika tidak ada transaksi sama sekali, kondisi ini bernilai *true.*

System.out.println("Belum ada transaksi pada rekening ini");
Jika *riwayatTransaksi* kosong, baris ini akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut.

} else {
Baris ini digunakan ketika kondisi *if* tidak terpenuhi. Artinya, *riwayatTransaksi* sudah memiliki setidaknya satu data transaksi, sehingga program akan menjalankan kode yang ada di dalam *else*.

System.out.println("--- MUTASI REKENING ---");
Baris ini menampilkan judul --- MUTASI REKENING --- sebagai penanda bahwa data yang akan ditampilkan berikutnya adalah daftar riwayat transaksi rekening.

for (Transaksi_2511532024 transaksi : riwayatTransaksi) {
Baris ini menggunakan *for-each* untuk mengambil setiap data transaksi yang terdapat di dalam *riwayatTransaksi* satu per satu.

Transaksi_2511532024 menunjukkan bahwa setiap data dalam list berupa objek dari *class* tersebut. Variabel transaksi digunakan untuk menyimpan satu objek transaksi yang sedang diproses. Tanda : dapat dibaca sebagai "dari", sehingga maksudnya adalah mengambil setiap transaksi dari *riwayatTransaksi.*

transaksi.cetakDetail();
Baris ini memanggil *method cetakDetail()* dari objek *transaksi.* Method tersebut digunakan untuk menampilkan detail dari transaksi yang sedang diproses. Karena berada di dalam perulangan *for*, setiap transaksi yang ada di *riwayatTransaksi* akan ditampilkan detailnya satu per satu.

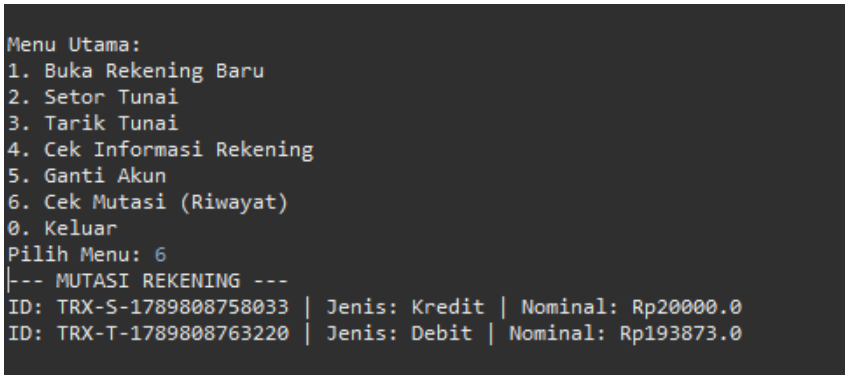
b. Class Main

case 6:
Baris ini menunjukkan pilihan menu ke-6 dalam struktur *switch-case* yaitu cek mutasi.

akunAktif.cetakMutasi();
Baris ini memanggil *method cetakMutasi()* dari objek *akunAktif. akunAktif* merupakan rekening yang sedang digunakan oleh pengguna, sedangkan *cetakMutasi()* digunakan untuk menampilkan seluruh riwayat transaksi dari rekening tersebut.

break;
break digunakan untuk menghentikan proses pada *case 6* setelah *cetakMutasi()* selesai dijalankan. Program kemudian keluar dari struktur *switch* dan melanjutkan ke bagian kode berikutnya.

Output:


Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 6
--- MUTASI REKENING ---
ID: TRX-S-1789808758033 | Jenis: Kredit | Nominal: Rp20000.0
ID: TRX-T-1789808763220 | Jenis: Debit | Nominal: Rp193873.0

PRAKTIKUM 3

A. Langkah-Langkah

1. Mengunci Entitas Transaksi

a. Buat Class Transaksi

public class Transaksi_2511532024 {
Membuat *class* bernama *Transaksi_2511532024*. *Class* ini digunakan untuk menyimpan dan mengelola data transaksi pada sistem perbankan.

private String idTransaksi;
Membuat atribut *idTransaksi* dengan tipe data *String* untuk menyimpan *ID* atau kode transaksi. *private* digunakan agar atribut tidak dapat diakses atau diubah secara langsung dari luar *class.*

private String jenis;
Membuat atribut jenis dengan tipe data *String* yang digunakan untuk menyimpan jenis transaksi, seperti *Kredit* untuk setor tunai dan *Debit* untuk tarik tunai.

private double nominal;
Membuat atribut nominal dengan tipe data *double* untuk menyimpan jumlah uang dalam suatu transaksi. Atribut dibuat *private* agar data tidak dapat dimanipulasi secara langsung dari luar *class.*

b. Constructor

public Transaksi_2511532024(String id, String jenis, double nominal) {
Membuat *constructor Transaksi_2511532024* yang digunakan untuk memberikan nilai awal pada objek transaksi ketika objek dibuat. *Constructor* menerima tiga parameter, yaitu *id*, jenis, dan

nominal.

this.idTransaksi = id;
Memasukkan nilai parameter id ke dalam atribut *idTransaksi*. Kata this menunjukkan bahwa *idTransaksi* merupakan atribut milik objek yang sedang dibuat.

this.jenis = jenis;
Memasukkan nilai parameter jenis ke dalam atribut jenis pada objek transaksi.

this.nominal = nominal;
Memasukkan nilai parameter nominal ke dalam atribut nominal pada objek transaksi.

c. Getter ID Transaksi

public String getIdTransaksi() {
Membuat *method getIdTransaksi()* yang digunakan untuk mengambil atau membaca nilai atribut *idTransaksi. Method* dibuat *public* agar dapat dipanggil dari *class* lain.

return idTransaksi;
Mengembalikan nilai yang tersimpan pada atribut *idTransaksi.*

d. Getter Jenis Transaksi

public String getJenis() {
Membuat *method getJenis()* yang digunakan untuk mengambil nilai dari atribut jenis.

return jenis;
Mengembalikan nilai yang tersimpan pada atribut jenis.

e. Getter Nominal

public double getNominal() {
Membuat method *getNominal()* yang digunakan untuk mengambil nilai dari atribut nominal.

return nominal;
Mengembalikan nilai nominal transaksi yang tersimpan pada objek.

f. Method Cetak Detail

public void cetakDetail() {
Membuat method *cetakDetail()* yang digunakan untuk menampilkan informasi lengkap dari suatu transaksi. Tipe *void* menunjukkan bahwa *method* ini tidak mengembalikan nilai.

System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
Menampilkan ID transaksi, jenis transaksi, dan nominal transaksi ke *console.* Tanda + digunakan untuk menggabungkan teks dengan nilai yang terdapat pada masing-masing atribut.

2. Mengunci Entitas Rekening dan Menambahkan PIN

n. Mengunci Atribut dengan Private

private String nomorRekening;
Membuat atribut *nomorRekening* dengan tipe *data String* untuk menyimpan nomor rekening. Atribut dibuat *private* agar tidak dapat diakses atau diubah secara langsung dari luar *class.*

private String namaPemilik;
Membuat atribut *namaPemilik* dengan tipe data *String* untuk menyimpan nama pemilik rekening.

private double saldo;
Membuat atribut *saldo* dengan tipe data *double* untuk menyimpan jumlah saldo yang dimiliki oleh rekening.

private String pin;
Membuat atribut *pin* dengan tipe data *String* untuk menyimpan PIN rekening. Atribut dibuat *private* agar PIN tidak dapat diakses secara langsung dari luar *class.*

ArrayList<Transaksi_2511532024> riwayatTransaksi;
Membuat *ArrayList* bernama *riwayatTransaksi* untuk menyimpan banyak objek *Transaksi_2511532024. ArrayList* ini digunakan untuk menyimpan seluruh riwayat transaksi yang dilakukan oleh suatu rekening.

o. Modifikasi Constructor untuk Menerima PIN Awal

public Rekening_2511532024(String nomor, String nama, double saldoAwal, String pinAwal) {
Membuat *constructor Rekening_2511532024* yang digunakan untuk memberikan nilai awal ketika objek rekening dibuat. *Constructor* menerima empat parameter, yaitu nomor rekening, nama pemilik, saldo awal, dan PIN awal.

this.nomorRekening = nomor;
Memasukkan nilai parameter nomor ke dalam atribut *nomorRekening.*

this.namaPemilik = nama;
Memasukkan nilai parameter nama ke dalam atribut *namaPemilik.*

this.saldo = saldoAwal;

Memasukkan nilai parameter *saldoAwal* ke dalam atribut saldo.

p. Validasi PIN di dalam Constructor

if (pinAwal.length() == 6) {
Memeriksa panjang PIN menggunakan *length().* Jika jumlah karakter pada *pinAwal* sama dengan 6, maka PIN dianggap memenuhi ketentuan.

this.pin = pinAwal;
Jika PIN memiliki panjang 6 karakter, nilai *pinAwal* disimpan ke dalam atribut *pin.*

} else {
Bagian *else* dijalankan jika panjang PIN tidak sama dengan 6 karakter.

System.out.println("Peringatan: PIN harus 6 digit! menggunakan PIN default 123456");
Menampilkan pesan peringatan bahwa PIN harus terdiri dari 6 digit. Jika PIN yang dimasukkan tidak memenuhi ketentuan, program akan menggunakan PIN default 123456.

this.pin = "123456";
Mengatur nilai atribut pin menjadi "123456" sebagai PIN *default* ketika PIN awal tidak memiliki 6 karakter.

this.riwayatTransaksi = new ArrayList<>();
Membuat objek *ArrayList* baru untuk menyimpan riwayat transaksi rekening. Setiap rekening memiliki tempat penyimpanan riwayat transaksinya sendiri.

System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
Menampilkan pesan bahwa rekening berhasil dibuat dan menampilkan nama pemilik rekening. Tanda + digunakan untuk menggabungkan teks dengan nilai *namaPemilik.*

q. Getter untuk Atribut yang diizinkan dibaca publik

public String getNomorRekening() {
Membuat *method getNomorRekening()* yang digunakan untuk mengambil atau membaca nilai atribut *nomorRekening.*

return nomorRekening;
Mengembalikan nilai nomor rekening yang tersimpan pada atribut *nomorRekening.*

public String getNamaPemilik() {
Membuat *method getNamaPemilik()* yang digunakan untuk mengambil atau membaca nama pemilik rekening.

return namaPemilik;
Mengembalikan nilai yang tersimpan pada atribut *namaPemilik.*

r. Method Otentifikasi Internal (Validasi Enkapsulasi)

public boolean otentikasi(String inputPin) {
Membuat *method otentikasi()* yang digunakan untuk memeriksa apakah PIN yang dimasukkan pengguna sesuai dengan PIN rekening. *Method* mengembalikan nilai *boolean*, yaitu true jika PIN benar dan false jika PIN salah.

return this.pin.equals(inputPin);
Membandingkan PIN yang tersimpan pada atribut pin dengan PIN

yang dimasukkan pengguna melalui *inputPin. Method equals()* digunakan untuk membandingkan isi dari dua *String*. Jika kedua PIN sama, hasilnya *true*, sedangkan jika berbeda hasilnya *false*.

B. Tugas/Latihan Praktikum

1. Menu 1 (Buka Rekening) : Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

   a. Class Main.

```
case 1:
System.out.print("Masukkan No Rekening: ");
String no = input.nextLine();

System.out.print("Masukkan Nama Pemilik: ");
String nama = input.nextLine();

System.out.print("Masukkan Saldo Awal: ");
double saldo = input.nextDouble();
input.nextLine();

System.out.print("Masukkan PIN (6 Digit): ");
String pin = input.nextLine();

if (pin.length() != 6) {
System.out.println("PIN harus terdiri dari 6 Digit!");
break;
}

Rekening_2511532024 rekeningBaru = new
Rekening_2511532024(no, nama, saldo, pin);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
System.out.println("Rekening berhasil ditambahkan.");
break;
```

Penjelasan Code:

a. Class Main

System.out.print("Masukkan PIN (6 Digit): ");
Menampilkan perintah kepada pengguna untuk memasukkan PIN yang terdiri dari 6 digit.

String pin = input.nextLine();
Membaca PIN yang dimasukkan pengguna menggunakan *nextLine()* dan menyimpannya ke dalam variabel pin dengan tipe data *String.*

if (pin.length() != 6) {
Memeriksa jumlah karakter PIN menggunakan *length().* Jika panjang PIN tidak sama dengan 6 karakter, maka kondisi *if* dijalankan.

System.out.println("PIN harus terdiri dari 6 Digit!");
Menampilkan pesan bahwa PIN yang dimasukkan harus terdiri dari 6 digit.

break;
Menghentikan proses pada *case* yang sedang dijalankan. Program kemudian kembali ke perulangan menu utama sehingga rekening tidak jadi dibuat.

Rekening_2511532024 rekeningBaru = new Rekening_2511532024(no, nama, saldo, pin);
Membuat objek rekening baru bernama *rekeningBaru* dari *class Rekening_2511532024.* Data yang diberikan adalah nomor rekening no, nama pemilik nama, saldo awal saldo, dan PIN pin.

daftarRekening.add(rekeningBaru);
Menambahkan objek *rekeningBaru* ke dalam *ArrayList*

daftarRekening, sehingga rekening tersebut tersimpan dalam daftar rekening.

akunAktif = rekeningBaru;
Menjadikan rekening yang baru dibuat sebagai *akunAktif*, yaitu rekening yang sedang digunakan oleh pengguna.

System.out.println("Rekening berhasil ditambahkan.");
Menampilkan pesan bahwa rekening berhasil ditambahkan ke dalam sistem.

break;
Menghentikan proses *case* setelah rekening berhasil dibuat dan ditambahkan. Program kemudian kembali ke menu utama.

Output

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 001
Masukkan Nama Pemilik: nisa
Masukkan Saldo Awal: 10000000
Masukkan PIN (6 Digit): 123456
Rekening atas nama nisa berhasil dibuat.
Rekening berhasil ditambahkan.
```

2. Keamanan Transaksi

   Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.

   - Panggil fungsi akunAktif.otentikasi(pinYangDiinput).
   - Jika mengembalikan nilai true, jalankan transaksi.

- Jika false, batalkan dan cetak: “Akses Ditolak: PIN yang anda masukkan salah!”.

a. Case 3

```
case 3:
if (akunAktif == null) {
System.out.println("Error: Mohon maaf, Anda belum
memiliki nomor rekening!");
} else {
System.out.print("Masukkan PIN: ");
String pinTarik = input.nextLine();
if (akunAktif.otentikasi(pinTarik)) {
System.out.print("Masukkan nominal tarik tunai: ");
double tarik = input.nextDouble();
input.nextLine();

akunAktif.tarikTunai(tarik); // Memanggil Behavior /
method
} else {
System.out.println("Akses ditolak: PIN yang anda
masukkan salah!");
}
}
break;
```

b. Case 6

```
case 6:
System.out.print("Masukkan PIN: ");
String pinMutasi = input.nextLine();
if (akunAktif.otentikasi(pinMutasi)) {
akunAktif.cetakMutasi();
} else {
System.out.println("Akses ditolak PIN yang anda
masukkan salah!");
}
break;
```

Penjelasan Code:

a. Case 3

System.out.print("Masukkan PIN: ");
Meminta pengguna memasukkan PIN sebelum melakukan transaksi tarik tunai.

String pinTarik = input.nextLine();
Membaca PIN yang dimasukkan pengguna dan menyimpannya ke dalam variabel *pinTarik.*

if (akunAktif.otentikasi(pinTarik)) {
Memanggil *method otentikasi()* pada rekening aktif untuk memeriksa apakah PIN yang dimasukkan sesuai dengan PIN rekening. Jika hasilnya *true*, pengguna dapat melanjutkan transaksi.

System.out.print("Masukkan nominal tarik tunai: ");
Meminta pengguna memasukkan jumlah uang yang ingin ditarik.

double tarik = input.nextDouble();
Membaca nominal tarik tunai yang dimasukkan pengguna dan menyimpannya ke dalam variabel tarik dengan tipe data *double.*

input.nextLine();
Membersihkan sisa karakter Enter setelah penggunaan *nextDouble(),* sehingga input berikutnya dapat dibaca dengan benar.

akunAktif.tarikTunai(tarik); // Memanggil Behavior / method
Memanggil *method tarikTunai()* dari objek *akunAktif* dengan nilai tarik sebagai parameter. *Method* ini digunakan untuk menjalankan

proses penarikan tunai.

} else {
Bagian ini dijalankan jika hasil *otentikasi()* adalah *false*, yang berarti PIN yang dimasukkan tidak sesuai.

System.out.println("Akses ditolak: PIN yang anda masukkan salah!");
Menampilkan pesan bahwa akses transaksi ditolak karena PIN yang dimasukkan salah.

break;
Menghentikan proses *case* 3 dan kembali ke menu utama.

b. Case 6.

System.out.print("Masukkan PIN: ");
Meminta pengguna memasukkan PIN sebelum dapat melihat riwayat atau mutasi transaksi rekening.

String pinMutasi = input.nextLine();
Membaca PIN yang dimasukkan pengguna dan menyimpannya ke dalam variabel *pinMutasi.*

if (akunAktif.otentikasi(pinMutasi)) {
Memanggil *method otentikasi()* untuk memeriksa apakah PIN yang dimasukkan sesuai dengan PIN rekening aktif. Jika benar, program akan melanjutkan ke proses mencetak mutasi.

akunAktif.cetakMutasi();

Memanggil method *cetakMutasi()* pada rekening aktif untuk menampilkan riwayat transaksi yang tersimpan dalam rekening tersebut.

} else {
Bagian *else* dijalankan apabila PIN yang dimasukkan tidak sesuai dengan PIN rekening.

System.out.println("Akses ditolak PIN yang anda masukkan salah!");
Menampilkan pesan bahwa akses untuk melihat mutasi ditolak karena PIN yang dimasukkan salah.

break;
Menghentikan proses *case* 6 dan mengembalikan program ke menu utama.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik tunai: 100000
Tarik tunai Rp100000.0 berhasil. Saldo saat ini: Rp900000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 3
Masukkan PIN: 111111
Akses ditolak: PIN yang anda masukkan salah!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 6
Masukkan PIN: 123456
--- MUTASI REKENING ---
ID: TRX-T-1790514328422 | Jenis: Debit | Nominal: Rp100000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
0. Keluar
Pilih Menu: 6
Masukkan PIN: 111111
Akses ditolak PIN yang anda masukkan salah!
```

PRAKTIKUM 4

A. Langkah-Langkah

1. Memastikan Akses Modifier di Rekening.java

   a. Buat Class Rekening

private String nomorRekening;
Membuat atribut *nomorRekening* dengan tipe data *String* untuk menyimpan nomor rekening. *Modifier private* digunakan agar atribut tidak dapat diakses secara langsung dari luar *class.*

private String namaPemilik;
Membuat atribut *namaPemilik* dengan tipe data *String* untuk menyimpan nama pemilik rekening.

private String pin;
Membuat atribut *pin* dengan tipe data *String* untuk menyimpan PIN rekening. *Modifier private* digunakan agar PIN tidak dapat diakses secara langsung dari luar *class.*

   b. Protected untuk mengakses Subclass langsung

protected double saldo;
Membuat atribut *saldo* dengan tipe data *double* untuk menyimpan jumlah saldo rekening. *Modifier protected* memungkinkan atribut diakses oleh *class* turunan.

protected ArrayList<Transaksi_2511532024> riwayatTransaksi;
Membuat *ArrayList* bernama *riwayatTransaksi* untuk menyimpan banyak objek *Transaksi_2511532024* sebagai riwayat transaksi rekening.

2. Membuat Subclass RekeningTabungan

public class RekeningTabungan extends Rekening_2511532024 {
Membuat class *RekeningTabungan* yang merupakan turunan dari *class Rekening_2511532024* menggunakan *keyword extends.*

private double sukuBunga;
Membuat atribut *sukuBunga* dengan tipe data *double* untuk menyimpan *persentase* bunga rekening tabungan.

public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
Membuat *constructor RekeningTabungan* untuk memberikan nilai awal berupa nomor rekening, nama pemilik, saldo awal, PIN, dan suku bunga.

super(nomor, nama, saldoAwal, pinAwal);
Memanggil *constructor* dari class induk *Rekening_2511532024* untuk menginisialisasi data nomor rekening, nama, saldo awal, dan PIN. Modul menjelaskan bahwa *super()* digunakan untuk memanggil *constructor* induk.

this.sukuBunga = sukuBunga;
Memasukkan nilai parameter *sukuBunga* ke dalam atribut *sukuBunga* milik objek.

public void tambahBungaAkhirBulan() {
Membuat method *tambahBungaAkhirBulan()* untuk menjalankan proses penambahan bunga pada rekening tabungan.

*double nominalBunga = saldo * (sukuBunga / 100);*
Menghitung nominal bunga berdasarkan saldo rekening dan persentase suku bunga

saldo += nominalBunga;
Menambahkan nominal bunga yang diperoleh ke dalam saldo rekening.

String idTrx = "TRX-B-" + System.currentTimeMillis();
Membuat *ID* transaksi bunga dengan menggabungkan teks *"TRX-B-"* dan waktu saat transaksi dibuat.

riwayatTransaksi.add(new Transaksi_2511532024(idTrx, "Bunga", nominalBunga));
Menambahkan transaksi bunga ke dalam *riwayatTransaksi* sebagai catatan transaksi rekening.

System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
Menampilkan informasi bahwa bunga berhasil ditambahkan beserta persentase dan nominal bunganya.

3. Membuat Subclass RekeningGiro

public class RekeningGiro extends Rekening_2511532024 {
Baris ini digunakan untuk membuat *class* baru bernama *RekeningGiro.* Kata *extends* menunjukkan bahwa *RekeningGiro* merupakan turunan dari *class Rekening_2511532024*. Artinya, *RekeningGiro* dapat mewarisi atribut dan *method* yang dimiliki *oleh Rekening_2511532024.*

private double batasOverdraft;
Baris ini membuat atribut bernama *batasOverdraft* dengan tipe data *double*. Atribut tersebut digunakan untuk menyimpan nilai batas *overdraft* pada rekening giro. Kata *private* berarti atribut ini hanya dapat diakses secara langsung dari dalam *class RekeningGiro.*

public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
Baris ini merupakan constructor dari *class RekeningGiro. Constructor* digunakan untuk memberikan nilai awal ketika sebuah objek *RekeningGiro* dibuat. *Constructor* ini menerima beberapa parameter, yaitu nomor untuk nomor rekening, nama untuk nama pemilik, *saldoAwal* untuk saldo awal, *pinAwal* untuk *PIN*, dan *batasOverdraft* untuk menentukan batas overdraft rekening giro.

super(nomor, nama, saldoAwal, pinAwal);
Baris ini digunakan untuk memanggil *constructor* dari *class Rekening_2511532024* sebagai *superclass.* Nilai nomor, nama, *saldoAwal*, dan *pinAwal* dikirim ke *constructor superclass* agar data dasar rekening dapat diinisialisasi oleh *class* induknya. Dengan begitu, *RekeningGiro* tidak perlu menulis ulang proses inisialisasi data rekening dasar.

this.batasOverdraft = batasOverdraft;
Baris ini digunakan untuk memasukkan nilai parameter *batasOverdraft* ke atribut *batasOverdraft* milik objek *RekeningGiro. this.batasOverdraft* mengacu pada atribut yang berada di dalam *class*, sedangkan *batasOverdraft* yang berada setelah tanda sama dengan merupakan parameter yang diterima oleh *constructor.*

public double getBatasOverdraft() {
Baris ini membuat *method* bernama *getBatasOverdraft(). Method* tersebut bersifat *public*, sehingga dapat dipanggil dari luar class. Tipe *double* menunjukkan bahwa *method* ini akan mengembalikan nilai berupa angka desimal, yaitu nilai dari batas *overdraft.*

return batasOverdraft;
Baris ini mengembalikan nilai yang tersimpan di dalam atribut *batasOverdraft.* Jadi, ketika *method getBatasOverdraft()* dipanggil, program akan mengambil dan memberikan nilai batas *overdraft* rekening tersebut.

B. Tugas/Latihan Praktikum

1. Pembuatan Multi-Produk

   Modifikasi case 1: buka Rekening Baru di Main.java

   - Setelah user menginput saldo awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Gio Bisnis"
   - Jika user memilih tabungan, minta input sukubunga dalam persen, lalu instansiasi objek RekeningTabungan.
   - Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
   - Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclassnya).

   a. Class Main.

```
System.out.println("Pilih Produk: 1. Tabungan Umum | 2.
Giro Bisnis | 3. Rekening VIP");
System.out.print("Pilihan: ");
int produk = input.nextInt();
input.nextLine();

Rekening_2511532024 rekeningBaru;
if (produk == 1) {
System.out.print("Suku Bunga (%): ");
double sukuBunga = input.nextDouble();
input.nextLine();
rekeningBaru = new RekeningTabungan(no, nama, saldo, pin,
sukuBunga);
} else if (produk == 2) {
System.out.print("Masukkan Batas Overdraft: ");
```

```
double batasOverdraft = input.nextDouble();
input.nextLine();
rekeningBaru = new RekeningGiro(no, nama, saldo, pin,
batasOverdraft);
} else {
System.out.println("Pilihan produk tidak valid!");
break;
}

daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
System.out.println("Rekening berhasil ditambahkan.");
break;
```

Penjelasan Code:

a. Class Main

System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
Baris ini menampilkan pilihan jenis produk rekening yang dapat dipilih oleh pengguna. Pengguna diberikan tiga pilihan, yaitu 1 untuk Tabungan Umum, 2 untuk Giro Bisnis

System.out.print("Pilihan: ");
Baris ini menampilkan tulisan *"Pilihan: "* tanpa membuat baris baru. Program kemudian menunggu pengguna memasukkan nomor produk yang ingin dipilih.

int produk = input.nextInt();
Baris ini digunakan untuk membaca angka yang dimasukkan oleh pengguna menggunakan *nextInt()*. Angka tersebut disimpan ke dalam variabel produk yang bertipe int. Nilai inilah yang nantinya digunakan untuk menentukan jenis rekening yang akan dibuat.

input.nextLine();

Baris ini digunakan untuk mengambil sisa karakter *newline (Enter)* yang masih tersimpan setelah penggunaan *nextInt().* Ini dilakukan agar pembacaan input berikutnya menggunakan *nextLine()* tidak langsung dilewati oleh program.

Rekening_2511532024 rekeningBaru;
Baris ini mendeklarasikan sebuah variabel bernama *rekeningBaru* dengan tipe *Rekening_2511532024.* Variabel ini nantinya digunakan untuk menyimpan objek rekening yang dibuat, baik objek *RekeningTabungan* maupun *RekeningGiro.* Hal ini bisa dilakukan karena kedua *class* tersebut merupakan turunan dari *Rekening_2511532024.*

if (produk == 1) {
Baris ini melakukan pengecekan apakah pengguna memilih produk dengan nomor 1. Jika nilai produk sama dengan 1, maka program akan menjalankan kode di dalam blok *if* untuk membuat rekening tabungan.

System.out.print("Suku Bunga (%): ");
Jika pengguna memilih produk 1, baris ini meminta pengguna memasukkan nilai suku bunga rekening tabungan dalam bentuk persen.

double sukuBunga = input.nextDouble();
Baris ini membaca nilai suku bunga yang dimasukkan pengguna menggunakan *nextDouble().* Nilai tersebut kemudian disimpan ke dalam variabel sukuBunga yang bertipe *double.*

input.nextLine();
Baris ini kembali digunakan untuk membersihkan karakter *Enter* yang tersisa setelah *nextDouble(),* sehingga input berikutnya dapat

dibaca dengan benar.

rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
Baris ini membuat objek baru dari *class RekeningTabungan*. Data *no, nama, saldo, pin,* dan *sukuBunga* diberikan kepada *constructor RekeningTabungan.* Objek yang telah dibuat kemudian disimpan ke dalam variabel *rekeningBaru*.

} else if (produk == 2) {
Jika kondisi sebelumnya tidak terpenuhi, program memeriksa apakah pengguna memilih produk 2. Jika produk bernilai 2, program akan menjalankan kode untuk membuat rekening giro.

System.out.print("Masukkan Batas Overdraft: ");
Baris ini meminta pengguna memasukkan nilai batas *overdraft* untuk rekening giro. Batas *overdraft* merupakan batas penggunaan dana melebihi saldo yang tersedia sesuai aturan yang diterapkan pada rekening giro.

double batasOverdraft = input.nextDouble();
Baris ini membaca nilai batas *overdraft* yang dimasukkan pengguna dan menyimpannya ke dalam variabel *batasOverdraft* dengan tipe data *double.*

input.nextLine();
Baris ini digunakan untuk membersihkan karakter *Enter* yang tersisa setelah *nextDouble(),* sama seperti pada bagian input suku bunga sebelumnya.

rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);

Baris ini membuat objek baru dari *class RekeningGiro*. Nilai *no, nama, saldo, pin,* dan *batasOverdraft* dikirim ke *constructor RekeningGiro.* Objek tersebut kemudian disimpan ke variabel *rekeningBaru.*

} else {
Bagian else dijalankan apabila nilai produk bukan 1 dan bukan 2.

System.out.println("Pilihan produk tidak valid!");
Baris ini menampilkan pesan bahwa pilihan produk yang dimasukkan tidak valid.

break;
Baris ini menghentikan proses pada blok atau perulangan tempat kode tersebut berada. Dengan adanya break, program tidak akan melanjutkan ke proses penambahan rekening apabila pilihan produk tidak valid.

daftarRekening.add(rekeningBaru);
Baris ini menambahkan objek *rekeningBaru* ke dalam *daftarRekening.* Dengan demikian, rekening yang baru dibuat akan disimpan di dalam *ArrayList* dan dapat digunakan kembali oleh program.

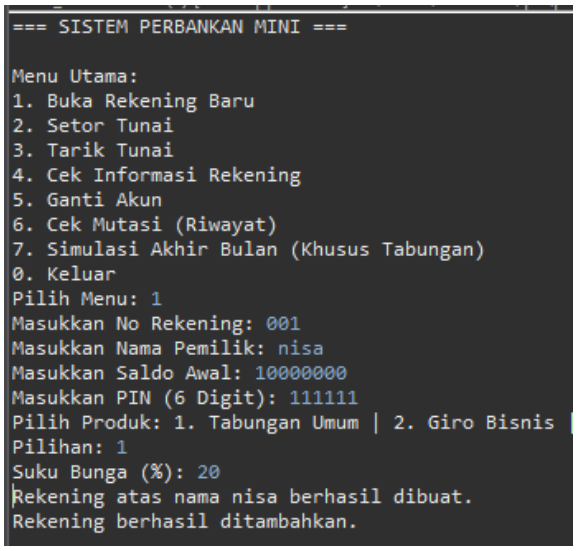
akunAktif = rekeningBaru;
Baris ini menjadikan rekening yang baru dibuat sebagai akun aktif. Artinya, setelah rekening berhasil dibuat, rekening tersebut langsung menjadi rekening yang sedang digunakan oleh pengguna.

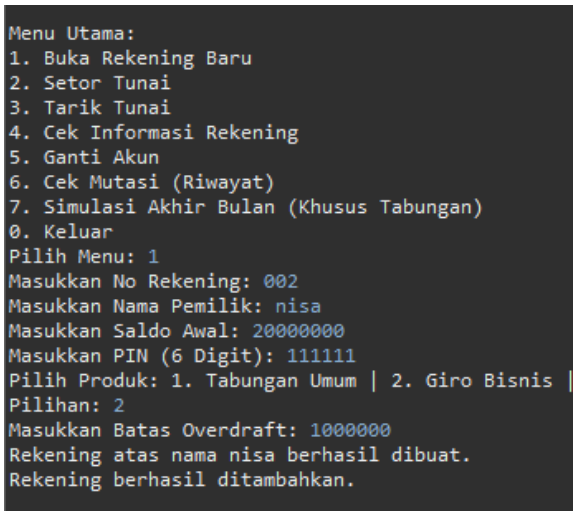
System.out.println("Rekening berhasil ditambahkan.");
Baris ini menampilkan pesan kepada pengguna bahwa rekening

telah berhasil dibuat dan ditambahkan ke dalam daftar rekening.

break;
Baris ini menghentikan proses pada perulangan atau menu yang sedang berjalan. Setelah rekening berhasil dibuat, program keluar dari proses tersebut sehingga tidak mengulangi pembuatan rekening.

Output





2. Simulasi Waktu Akhir Bulan
    - Tambahkan menu baru: “7. Simulasi Akhir Bulan (Khusus Tabungan)” di CLI

- Didalam case 7; periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
- Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
- Jika bukan, tolak dengan pesan: “Gagal: fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.”

a. Class Main

```
case 7:
    if (akunAktif instanceof RekeningTabungan ) {
    RekeningTabungan tab = (RekeningTabungan) akunAktif;
    tab.tambahBungaAkhirBulan();
    } else {
    System.out.println("Gagal: Fitur bunga akhir bulan
hanya berlaku untuk Rekening Tabungan");
    }
    break;
```

Penjelasan Code:

a. Class Main

case 7:
Baris ini menunjukkan bahwa kode berikutnya dijalankan ketika pengguna memilih menu nomor 7.

if (akunAktif instanceof RekeningTabungan) {
Baris ini mengecek apakah objek yang sedang tersimpan di dalam *akunAktif* merupakan objek dari *class RekeningTabungan*. *instanceof* digunakan untuk mengetahui tipe objek yang sedang digunakan. Jika *akunAktif* memang merupakan rekening tabungan, maka program akan menjalankan kode di dalam *if.*

RekeningTabungan tab = (RekeningTabungan) akunAktif;

Baris ini melakukan casting terhadap *akunAktif* menjadi tipe *RekeningTabungan,* kemudian hasilnya disimpan ke variabel *tab. Casting* diperlukan supaya objek tersebut dapat digunakan sebagai *RekeningTabungan* dan *method* khusus seperti *tambahBungaAkhirBulan()* dapat dipanggil.

tab.tambahBungaAkhirBulan();
Baris ini memanggil *method tambahBungaAkhirBulan()* yang terdapat pada *class RekeningTabungan*. *Method* tersebut digunakan untuk menghitung dan menambahkan bunga akhir bulan ke saldo rekening tabungan sesuai dengan aturan yang dibuat di dalam *method* tersebut.

} else {
Bagian *else* akan dijalankan apabila *akunAktif* bukan merupakan objek *RekeningTabungan.* Misalnya, akun aktif yang sedang digunakan adalah *RekeningGiro.* Karena fitur bunga akhir bulan khusus untuk rekening tabungan, proses penambahan bunga tidak dilakukan.

System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan");
Baris ini menampilkan pesan kepada pengguna bahwa proses gagal karena fitur bunga akhir bulan hanya dapat digunakan pada rekening tabungan. Jadi kalau akun aktifnya giro, program tidak akan memaksakan bunga ke rekening yang bukan tempatnya. Untuk sekali ini, program punya batasan yang masuk akal.

break;
Baris ini menghentikan proses pada case 7 dan keluar dari struktur switch. Setelah proses bunga selesai atau pesan kesalahan ditampilkan, program akan kembali ke bagian menu utama.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 7
Bunga 20.0% berhasil ditambahkan: Rp2000000.0
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu: 7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan
```

PRAKTIKUM 5

A. Langkah-Langkah

1. Refaktor Method tarikTunai pada Kelas Induk (Rekening.java)

   a. Mengubah void menjadi boolean

public boolean tarikTunai(double nominal) {
Membuat *method tarikTunai()* untuk menjalankan transaksi penarikan tunai. *Parameter nominal* menyimpan jumlah uang yang ingin ditarik, sedangkan tipe data *boolean* digunakan untuk mengembalikan nilai *true* atau *false* sesuai hasil transaksi.

if (nominal <= 0) {
Memeriksa apakah *nominal* penarikan kurang dari atau sama dengan nol. Jika kondisi terpenuhi, transaksi tidak dapat dilanjutkan karena *nominal* penarikan tidak valid.

System.out.println("Transaksi Gagal: Nominal tarik harus lebih dari 0!");
Menampilkan pesan bahwa *transaksi* penarikan gagal karena *nominal* yang dimasukkan harus lebih dari nol.

return false;
Mengembalikan nilai *false* untuk menandakan bahwa transaksi penarikan gagal dilakukan.

} else if (nominal > saldo) {
Memeriksa apakah *nominal* penarikan lebih besar daripada *saldo* yang tersedia. Jika kondisi terpenuhi, penarikan ditolak karena saldo rekening tidak mencukupi.

System.out.println("Transaksi Gagal: Saldo tidak mencukupi.

saldo anda Rp" + saldo);
Menampilkan pesan kegagalan transaksi beserta jumlah *saldo* yang tersedia pada rekening.

return false;
Mengembalikan nilai *false* untuk menunjukkan bahwa *transaksi* tidak berhasil karena *saldo* tidak mencukupi.

} else {
Menjalankan blok kode ketika nominal penarikan lebih dari nol dan saldo rekening mencukupi untuk melakukan transaksi.

saldo -= nominal;
Mengurangi *saldo* rekening sesuai dengan *nominal* uang yang ditarik. *Operato*r -= digunakan untuk mengurangi nilai saldo secara langsung.

String idTrx = "TRX-T-" + System.currentTimeMillis();
Membuat *ID* transaksi dengan awalan *TRX-T-* yang digabungkan dengan nilai waktu saat ini dalam milidetik. *ID* tersebut digunakan sebagai identitas transaksi penarikan.

Transaksi_2511532024 trxBaru = new Transaksi_2511532024(idTrx, "Debit", nominal);
Membuat objek transaksi bernama *trxBaru* dari *class Transaksi_2511532024*. Objek ini menyimpan *ID* transaksi, jenis transaksi berupa Debit, dan nominal uang yang ditarik.

riwayatTransaksi.add(trxBaru);
Menambahkan objek *trxBaru* ke dalam *riwayatTransaksi* agar transaksi penarikan tersimpan dalam daftar riwayat transaksi

rekening.

System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
Menampilkan pesan bahwa penarikan tunai berhasil dilakukan, disertai *nominal* uang yang ditarik dan jumlah saldo rekening setelah transaksi.

return false;
Mengembalikan nilai hasil transaksi melalui tipe data *boolean.* Namun, kode tersebut seharusnya menggunakan return true; karena transaksi telah berhasil dilakukan. Nilai *true* menunjukkan transaksi berhasil, sedangkan *false* menunjukkan transaksi gagal.

2. Melakukan Overriding pada RekeningGiro.java
   a. Implementasi Polimorfisme (Overriding)

@Override
Digunakan untuk menandai bahwa *method tarikTunai() pada class RekeningGiro* merupakan *method* yang menimpa *method* dengan nama yang sama pada kelas induknya, yaitu *Rekening.*

public boolean tarikTunai(double nominal) {
Membuat *method tarikTunai()* untuk menjalankan transaksi penarikan tunai pada rekening giro. Parameter *nominal* menyimpan jumlah uang yang ingin ditarik, sedangkan tipe data *boolean* digunakan untuk mengembalikan nilai *true* jika transaksi berhasil atau *false* jika transaksi gagal.

if (nominal <= 0) {
Memeriksa apakah nominal penarikan kurang dari atau sama dengan nol. Jika kondisi terpenuhi, transaksi tidak dapat dilakukan karena jumlah uang yang ingin ditarik tidak valid.

System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
Menampilkan pesan bahwa transaksi penarikan rekening giro gagal karena nominal yang dimasukkan harus lebih dari nol.

return false;
Mengembalikan nilai *false* untuk menandakan bahwa transaksi penarikan gagal karena nominal tidak memenuhi ketentuan.

b. Logika Khusus Giro

else if (nominal > (saldo + batasOverdraft)) {
Memeriksa apakah nominal penarikan melebihi jumlah saldo yang tersedia ditambah batas *overdraft.* Kondisi ini digunakan untuk memastikan penarikan tidak melampaui batas saldo negatif yang diizinkan pada rekening giro.

System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
Menampilkan pesan bahwa transaksi gagal karena nominal penarikan melebihi batas *overdraft.* Pesan tersebut juga menampilkan jumlah maksimal penarikan yang dapat dilakukan berdasarkan saldo dan batas *overdraft.*

return false;
Mengembalikan nilai *false* untuk menunjukkan bahwa transaksi penarikan gagal karena nominal yang dimasukkan melebihi batas *overdraft* rekening giro.

else {
Menjalankan blok kode apabila nominal penarikan lebih dari nol

dan tidak melebihi saldo ditambah batas *overdraft* yang diizinkan.

saldo -= nominal;
Mengurangi saldo rekening giro sesuai dengan nominal uang yang ditarik. Jika nominal penarikan lebih besar daripada saldo awal, saldo dapat menjadi negatif selama masih berada dalam batas *overdraft.*

String idTrx = "TRX-G-" + System.currentTimeMillis();
Membuat *ID* transaksi dengan awalan *TRX-G-* yang digabungkan dengan nilai waktu saat ini dalam milidetik. *ID* tersebut digunakan sebagai identitas transaksi penarikan pada rekening giro.

riwayatTransaksi.add(new Transaksi_2511532024(idTrx, "Debit Giro", nominal));
Membuat objek *Transaksi_2511532024* yang berisi *ID* transaksi, jenis transaksi Debit Giro, dan nominal penarikan. Objek tersebut kemudian ditambahkan ke dalam *riwayatTransaksi* agar transaksi tersimpan dalam riwayat rekening.

System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. saldo saat ini: Rp" + saldo);
Menampilkan pesan bahwa transaksi penarikan tunai rekening giro berhasil dilakukan, disertai nominal uang yang ditarik dan saldo rekening setelah transaksi.

return false;
Mengembalikan nilai berupa *false* sebagai hasil pemanggilan *method*. Namun, karena transaksi telah berhasil dilakukan, bagian ini seharusnya menggunakan return *true;* agar hasil transaksi menunjukkan keberhasilan.

public double getBatasOverdraft() {
Membuat *method getBatasOverdraft()* dengan tipe pengembalian *double* untuk mengambil nilai batas *overdraft* yang tersimpan pada atribut *batasOverdraft*. *Method* dibuat *public* agar dapat dipanggil dari *class* lain.

3. Uji Coba Pengikatan Dinamis Jalankan **Main.java**

   Coba buat akun baru dan Pilih Produk: 2. Giro. Beri saldo 100.000 dan batas *overdraft* 500.000. Coba tarik tunai 400.000. Jika ini Rekening biasa, pasti ditolak. Namun karena objeknya adalah *RekeningGiro,* Java akan otomatis memanggil logika di Langkah 2, dan saldo akan menjadi minus 300.000.

```
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 1
Masukkan No Rekening: 001
Masukkan Nama Pemilik: nisa
Masukkan Saldo Awal: 100000
Masukkan PIN (6 Digit): 111111
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis | 3. Rekening VIP
Pilihan: 2
Masukkan Batas Overdraft: 500000
Rekening atas nama nisa berhasil dibuat.
Rekening berhasil ditambahkan.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 3
Masukkan PIN: 111111
Masukkan nominal tarik tunai: 400000
Tarik tunai Giro Rp400000.0 berhasil. saldo saat ini: Rp-300000.0
```

B. Tugas/Latihan Praktikum

1. Buat Method Transfer : di dalam *class* **Rekening.java**, tambahkan satu *method bar* dengan nama:

Public void transfer(Rekening tujuan, double nominal, String pinInput)

- *Method* ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.
- Logika *method*
  - Pertama, lakukan ***otentikasi(pinInput).*** Jika *false*, batalkan.
  - Kedua, panggil ***this.tarikTunai(nominal).*** (*overriding*, jika *this* adalah Giro, ia otomatis menggunakan aturan *overdraft*).
  - Ketiga, periksa kembalian nilai *tarikTunai*. Jika *true*, panggil ***tujuan.setorTunai(nominal).***

a. Class Rekening

```
public void transfer(Rekening_2511532024 tujuan, double
nominal, String pinInput) {
if (!otentikasi(pinInput)) {
System.out.println("Transfer dibatalkan: PIN salah!");
return;
}

boolean berhasil = this.tarikTunai(nominal);
if (berhasil) {
tujuan.setorTunai(nominal);
System.out.println("Transfer Rp" + nominal + " ke
rekening" + tujuan.getNomorRekening() + " berhasil.");
} else {
System.out.println("Transfer Gagal: penarikan dari
rekening asal tidak berhasil.");
}
}
```

Penjelasan Code:

a. Class Rekening

public void transfer(Rekening_2511532024 tujuan, double nominal, String pinInput) {
Membuat *method transfer()* yang digunakan untuk melakukan transfer uang dari rekening asal ke rekening tujuan. *Method* ini menerima tiga parameter, yaitu tujuan sebagai objek rekening penerima, nominal sebagai jumlah uang yang akan ditransfer, dan pinInput sebagai PIN untuk melakukan autentikasi.

if (!otentikasi(pinInput)) {
Memeriksa apakah *PIN* yang dimasukkan pengguna sesuai dengan PIN rekening asal melalui pemanggilan *method otentikasi().* Tanda ! digunakan untuk membalik hasil kondisi, sehingga blok if dijalankan apabila PIN tidak sesuai.

System.out.println("Transfer dibatalkan: PIN salah!");
Menampilkan pesan bahwa proses transfer dibatalkan karena PIN yang dimasukkan pengguna salah.

return;
Menghentikan pelaksanaan *method transfer()* apabila PIN yang dimasukkan tidak sesuai. Dengan demikian, proses penarikan dan pengiriman uang tidak dilanjutkan.

boolean berhasil = this.tarikTunai(nominal);
Memanggil *method tarikTunai()* pada objek rekening asal untuk menarik uang sesuai nominal transfer. Hasil pemanggilan *method* disimpan dalam variabel berhasil bertipe *boolean.* Nilai *true* menunjukkan penarikan berhasil, sedangkan *false* menunjukkan penarikan gagal. Penggunaan this merujuk pada objek rekening yang sedang menjalankan *method transfer().*

if (berhasil) {
Memeriksa nilai variabel berhasil. Jika nilainya *true,* proses transfer dapat dilanjutkan dengan menambahkan uang ke rekening tujuan.

tujuan.setorTunai(nominal);
Memanggil *method setorTunai()* pada objek tujuan untuk menambahkan uang ke rekening penerima sesuai nominal *transfer* yang dilakukan.

System.out.println("Transfer Rp" + nominal + " ke rekening" + tujuan.getNomorRekening() + " berhasil.");
Menampilkan pesan bahwa transfer berhasil dilakukan. Pesan tersebut memuat nominal uang yang ditransfer dan nomor rekening tujuan yang diperoleh melalui *method getNomorRekening().*

} else {
Menjalankan blok *else* apabila nilai variabel berhasil adalah *false*, yang menunjukkan bahwa penarikan dari rekening asal tidak berhasil.

System.out.println("Transfer Gagal: penarikan dari rekening asal tidak berhasil.");
Menampilkan pesan bahwa transfer gagal karena proses penarikan uang dari rekening asal tidak berhasil dilakukan.

2. Integrasi ke CLI Utama: buka **Main.java** dan tambahkan menu **8. Transfer ke Rekening Lain.**
   - Minta pengguna memasukkan Nomor Rekening Tujuan.
   - Cari nomor tersebut di dalam **daftarRekening.**
   - Jika ketemu, minta nominal transfer dan PIN, lalu panggil ***akunAktif.transfer(akunTujuan, nominal, pin).***

```
case 8:
System.out.print("Masukkan Nomor Rekening Tujuan: ");
String noTujuan = input.nextLine();

Rekening_2511532024 akunTujuan = null;
for (Rekening_2511532024 r : daftarRekening) {
if (r.getNomorRekening().equals(noTujuan)) {
akunTujuan = r;
break;
}
}

if (akunTujuan == null) {
```

```
System.out.println("Rekening tujuan tidak ditemukan!");
} else if (akunTujuan == akunAktif) {
System.out.println("Tidak bisa transfer ke rekening
sendiri");
} else {
System.out.print("Masukkan nominal transfer: ");
double nominalTransfer =
Double.parseDouble(input.nextLine());
System.out.print("Masukkan PIN: ");
String pinTransfer = input.nextLine();

akunAktif.transfer(akunTujuan, nominalTransfer,
pinTransfer);
}
break;
```

Penjelasan Code:

case 8:
case 8: merupakan bagian dari struktur *switch* yang dijalankan ketika pengguna memilih menu nomor 8, yaitu fitur transfer rekening.

System.out.print("Masukkan Nomor Rekening Tujuan: ");
menampilkan pesan kepada pengguna agar memasukkan nomor rekening tujuan. Perintah *print()* membuat kursor tetap berada pada baris yang sama setelah pesan ditampilkan.

String noTujuan = input.nextLine();
Variabel *noTujuan* bertipe data *String* digunakan untuk menyimpan nomor rekening tujuan yang dimasukkan oleh pengguna. *input.nextLine()* membaca input tersebut sebagai teks.

Rekening_2511532024 akunTujuan = null;
Kode ini mendeklarasikan variabel *akunTujuan* bertipe objek *Rekening_2511532024* untuk menyimpan rekening tujuan yang ditemukan. Nilai awal *null* menunjukkan bahwa belum ada objek rekening yang disimpan.

for (Rekening_2511532024 r : daftarRekening) {

Perulangan *for-each* digunakan untuk memeriksa setiap objek rekening yang terdapat dalam *daftarRekening.* Pada setiap perulangan, variabel r mewakili satu objek rekening yang sedang diperiksa.

if (r.getNomorRekening().equals(noTujuan)) {
Kondisi *if* membandingkan nomor rekening milik objek r, yang diperoleh melalui *getNomorRekening(),* dengan nomor rekening yang dimasukkan pengguna pada variabel *noTujuan*. *Method equals()* digunakan untuk memeriksa apakah kedua teks tersebut sama.

akunTujuan = r;
Jika nomor rekening cocok, objek rekening yang sedang diperiksa disimpan ke dalam variabel *akunTujuan.* Dengan demikian, program dapat menggunakan objek tersebut sebagai rekening penerima transfer.

break;
Perintah *break* digunakan untuk menghentikan perulangan setelah rekening tujuan ditemukan. Hal ini mencegah program melanjutkan pencarian ke rekening lainnya.

if (akunTujuan == null) {
Kondisi ini memeriksa apakah variabel *akunTujuan* masih bernilai *null*. Jika iya, berarti program tidak menemukan rekening yang sesuai dengan nomor rekening yang dimasukkan pengguna.

System.out.println("Rekening tujuan tidak ditemukan!");
Perintah ini menampilkan pesan bahwa rekening tujuan tidak ditemukan. *println()* menampilkan teks dan memindahkan kursor ke baris berikutnya.

else if (akunTujuan == akunAktif) {
Kondisi *else if* dijalankan jika rekening tujuan ditemukan. *Operator* == memeriksa apakah *akunTujuan* dan *akunAktif* merujuk pada objek rekening yang sama.

System.out.println("Tidak bisa transfer ke rekening sendiri");
Kode ini menampilkan pesan bahwa pengguna tidak diperbolehkan melakukan transfer ke rekening yang sedang digunakan sebagai rekening aktif.

else {
Blok *else* dijalankan jika rekening tujuan ditemukan dan bukan objek rekening yang sedang aktif. Program kemudian melanjutkan proses dengan meminta nominal transfer dan PIN pengguna.

System.out.print("Masukkan nominal transfer: ");
Kode ini menampilkan pesan agar pengguna memasukkan jumlah uang yang akan ditransfer ke rekening tujuan.

double nominalTransfer = Double.parseDouble(input.nextLine());
Variabel *nominalTransfer* bertipe data *double* digunakan untuk menyimpan jumlah uang yang akan ditransfer. *input.nextLine()* membaca input sebagai teks, sedangkan *Double.parseDouble()* mengubah teks tersebut menjadi nilai numerik bertipe *double.*

System.out.print("Masukkan PIN: ");
Kode ini menampilkan pesan agar pengguna memasukkan PIN sebagai bagian dari proses autentikasi sebelum transfer dilakukan.

String pinTransfer = input.nextLine();

Variabel *pinTransfer* bertipe data *String* digunakan untuk menyimpan PIN yang dimasukkan pengguna. Nilai tersebut nantinya dikirim ke *method transfer()* untuk diperiksa.

akunAktif.transfer(akunTujuan, nominalTransfer, pinTransfer);
Kode ini memanggil *method transfer()* dari objek rekening aktif. Method tersebut menerima tiga argumen, yaitu objek rekening tujuan *(akunTujuan),* jumlah uang yang ditransfer *(nominalTransfer),* dan PIN pengguna *(pinTransfer).* Proses autentikasi PIN, penarikan saldo dari rekening asal, serta penambahan saldo ke rekening tujuan dilakukan di dalam *method* tersebut.

break;
Perintah *break* mengakhiri eksekusi *case 8* sehingga program keluar dari struktur *switch* dan tidak menjalankan *case* berikutnya.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu: 8
Masukkan Nomor Rekening Tujuan: 1002
Masukkan nominal transfer: 50000
Masukkan PIN: 111111
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp450000.0
Setor tunai Rp50000.0 berhasil. Saldo saat ini: Rp250000.0
Transfer Rp50000.0 ke rekening1002 berhasil.
```

Link Video Penjelasan:

https://youtu.be/_fnzWhrLHpU